

Project Report

Wildlife Information System (WIS)

Document Number: WIS-SRS-001
Version: 2.0
Date: June 2026

Prepared by:
Muhammad Ehtisham
Fahad Khan
Muhammad Abbas

Table of Contents

1. INTRODUCTION	
1.1 Background	3
1.2 Existing System	3
1.3 Issues & Motivation	4
1.4 Problem Statement	4
1.5 Scope	4
1.6 Project Management	5
1.7 Technological Overview	6
1.8 Goals & Responsibilities	6
1.9 Process Model	6
1.10 Project Management System	7
2. SOFTWARE REQUIREMENTS SPECIFICATION	8
2.1 Types of Requirements	8
2.2 Hardware Requirements	8
2.3 Software Requirements	9
2.4 Functional Requirements	10
2.5 Non-Functional Requirements	12
2.6 Use Cases	13
3. ANALYSIS	23
3.1 Domain Model	23
3.2 Data Flow Diagram (DFD)	24
3.3 System Sequence Diagrams	26
3.4 Activity Diagram	28
3.5 Database ER Diagram	29
4. DESIGN	30
4.1 Class Diagram	30
4.2 Sequence Diagram	31
4.3 Architecture	32
4.4 Interface Design	34
5. IMPLEMENTATION	37
6. TESTING	38
6.1 Acceptance Testing	38
6.2 Test Cases	38
6.3 Test Plan	39

1. INTRODUCTION

1.1 Background

Wildlife means animals that live naturally in forests, mountains, deserts and in other environments like elephants, lions, wolves, dogs etc. These animals are very important because they help to maintain the balance of nature. For example: predators such as wolves control the population of other animals, and the birds help plants to grow by spreading seeds and pollinating flowers. Now, much information about wildlife is collected around the world. This information includes Animal, population number, Movement and migration routes, Animal behavior, DNA and genetic information, Health and disease records. However, the data is stored in many different places. Different organizations, universities, researchers, and government departments collect data separately. There is no single system where all wildlife information can be found. In Pakistan and many other countries, wildlife data is usually stored in reports, spreadsheets, notebooks, and paper records. Because of this, it becomes difficult for researchers, Doctors, and government agencies to access and use the information.

To solve this problem, Our System the Wildlife Information System (WIS) is proposed. WIS is a centralized system that stores all wildlife data in one organized database. Users can easily access and share information through an API, making wildlife research and conservation more efficient.

1.2 Existing System

Currently, wildlife data in most countries is managed in one of the following ways:

- Government forest departments maintain their own paper base or basic spreadsheet or excel file records of wildlife populations and incidents.
- Research universities collect data during research or field trips and store it in local databases that are not accessible to the public.
- International conservation organizations like WWF and IUCN maintain global databases, but they are limited and they have not included local or regional level details.
- Some IoT based systems track animal movement using GPS collars, but this data is kept private and not shared in public platforms.

The major weakness of all these approaches is that they do not talk to each other. There is no standard system that collects, organizes, and shares wildlife data openly.

1.3 Issues & Motivation

The following problems motivate the development of the Wildlife Information System:

- **Data Isolation:** Wildlife data exists in silos. Researchers, forest departments, and NGOs each have their own records but cannot share them.
- **No Standard Format:** Data is stored in different formats like Excel files, PDF reports, handwritten notes etc. we are making it impossible to combine or compare across organizations.
- **Lack of Open Access or Source:** Most wildlife databases are closed. Researchers must physically contact organizations and wait weeks to get data they need for their studies.
- **Missed Opportunities for Analysis:** Because data is fragmented, important patterns such as disease outbreaks, migration route changes, or population decline are often noticed too late.
- **Waste of Research Resources:** Multiple researchers sometimes collect the same data independently because they do not know it already exists elsewhere. Which waste their time.

1.4 Problem Statement

There is no single system in Pakistan where all the wildlife information is stored or shared. The different universities, research institutions and government institutions have but they keep their own records separately or private, because of this. It is most difficult to find information about animals like their populations, their behavior, DNA and Their Health. This creates many problems Researchers may collect same data repeatedly because they don't know it already exists. And also, the most important information can be missed, and govt agencies may take decisions without complete or accurate data.

The Wildlife Information system solves these problems. It will store wildlife information in one central database. And Authorized users can access this information through secure API and they can use it for research, wildlife monitoring, conservation projects and they can use for decision making.

1.5 Scope

The Wildlife Information System will cover the following:

- It will keep all the wildlife information in one central database. The information will include like animal profile, Population, movement tracking, DNA records, health, their behavior and environment data.
- There will be a backend system that will validate, organize, and manages all incoming data from different sources, like field researchers, IoT tracking devices, and lab results.
- It will include an API. A RESTful API layer that allows external users and applications to send queries and receive structured wildlife data in standard format like JSON.
- The User roles include Data Entry Operators, Researchers, Government Officials, and System Administrators, each will have different level of access.
- There will be a basic web interface in which can search and can enter data manual.

The following are explicitly outside the scope of WIS:

- This system will not install GPS collar or tracking device, manufacturing, or design. It will only store data transmitted from existing devices.
- Real-time live animal tracking maps or animated movement visualization on a geographic interface (this may be a future Phase 2 extension).
- This system will not provide any real time data or tracking or animated maps showing animal movements. But this may be a future plan in version 2.0 extension
- This system will not provide any mobile application for android or iOS. But the other developers may build these separately using the WIS public API.
- This system will not use AI or ML for identifying animal from images or sensor data. This system stores records but does not perform intelligent pattern recognition.
- This system will not connect directly with international database like IUCN Red list or Cities in v1.0 but may connect in later versions.
- This system will not include financial, HR, payroll, or administrative management features. It's mainly focuses on ecological and animal data.
- This system will not include offline data collection tools or special devices for field use in remote areas where the internet is not available.
- This system will not provide automatic alerts for diseases outbreaks, environmental risks or illegal hunting etc.
- This system will not support multiple languages. this system will operate in English only in v1.0.
- The system will not handle legal reporting wildlife permits or enforcement tracking systems.

1.6 Project Management

This project will be managed using a structured approach with clearly defined phases, responsibilities, and timelines.

Phase	Activity	Duration
Phase 1	Requirements gathering and analysis	2 weeks
Phase 2	System design (DB schema, API design)	2 weeks
Phase 3	Backend development and database setup	3 weeks
Phase 4	API development and testing	2 weeks
Phase 5	Documentation and final review	1 week

The Tools used: GitHub for version control, WhatsApp and Discord for team communication, and Google Docs for shared documentation. Weekly meetings every Saturday to review progress.

1.7 Technological Overview

Component	Technology
Backend Framework	Node.js with Express.js
Database	PostgreSQL (relational, for structured data)
API Type	RESTful API (JSON responses)
Authentication	JWT (JSON Web Tokens)
Frontend (basic)	HTML, CSS, JavaScript
Version Control	Git / GitHub
Hosting (future)	Cloud (AWS or DigitalOcean)
Data Format	JSON and CSV for exports

1.8 Goals & Responsibilities

Primary Goal: To build a centralized wildlife information database that is accurate, well organized, and accessible through an API for research and analysis purposes.

Secondary Goals:

- Encourage data sharing among wildlife organizations.
- Provide a platform that supports conservation and research decision making.
- Build a scalable foundation that can grow with more data and users over time.

1.9 Process Model

This project will follow an Incremental Development Model. which means the system is built step by step approach, where each step adds new functionality and improve previous. This model was chosen because:

- Requirements are mostly clear, but small changes are expected during development.
- It allows early testing of core features like the database and API before the full system is complete.
- Each increment produces a working version that can be reviewed and improved.

The three increments are:

- Increment 1: Database design and basic data entry system.
- Increment 2: API development and user authentication.
- Increment 3: Search, filtering, export, and final testing.

1.10 Project Management System

The team will use GitHub Projects to manage the work of the Project. All Tasks will be organized in a board format with three columns:

1. **To Do:** The tasks that need to be started
2. **In Progress:** The tasks that are currently being worked on.
3. **Done:** The tasks that have been completed

Each team member is assigned specific issues or tasks on the GitHub repository. Which will help us in tracking progress and make sure every task is completed on Time.

2. SOFTWARE REQUIREMENTS SPECIFICATION

2.1 Types of Requirements

Software requirements are divided into two main types (i) Functional Requirements (ii) Non Functional Requirements. Functional requirements describe what the system should do, and Non-functional requirements describe how the system should behave in terms of performance, security, and reliability.

User Classes & Roles

Role	Access Level	Capabilities
Admin	Full	Supervise user accounts, perform CRUD on all entities, and modify configurations.
Data Entry Operator	Write + Read	Create or modify profiles for wildlife, including tracking, medical, and genetic data.
Researcher	Read + Export	Execute queries via portal/API and download records in CSV or JSON formats.
Government Official	Read Only	Limited to viewing and searching existing animal information and profiles.
Developer	API Read	Retrieve data from public endpoints using authenticated JWT tokens.

Assumptions & Dependencies

- Constant network connectivity is assumed for all users interacting with the web portal or API.
- WIS acts strictly as a storage repository for GPS data generated by external tracking hardware.
- The initial release (v1.0) will support the English language exclusively for all operations.

2.2 Hardware Requirements

Component	Minimum Specification
Processor	Intel Core i3 (or equivalent), 2.0 GHz
RAM	4 GB (8 GB recommended for development)
Storage	50 GB free disk space for database
Network	Stable internet connection for API access
Server (deployment)	2 CPU cores, 4 GB RAM, 100 GB SSD (cloud instance)

2.3 Software Requirements

Software	Purpose
Windows 10 / Ubuntu 20.04	Operating System
Node.js v18+	Backend runtime
PostgreSQL 14+	Database management
Postman	API testing
VS Code	Code editor
Git	Version control
Browser (Chrome/Firefox)	Frontend testing

2.4 Functional Requirements

These functional requirements describes what the system must be able to do:

FR-01: Animal Profile Management

Users need to be able to put in new wildlife records and also change or remove them when something comes up. Details like species age and weight should get captured in there too. The system creates a unique ID by itself for each animal. I think checking that the entered data is valid has to happen as well. It feels like those parts connect but maybe not in a super strict order.

FR-02: Movement Data Recording

This function stores GPS coordinates and timestamps tied to animal IDs. It seems batch uploads from a CSV file are how multiple entries get added at once. The coordinates get checked before anything saves to confirm they are valid. That part feels important but some details on the verification step are not totally clear right away. Maybe the checks happen in a specific order that is easy to miss.

FR-03: DNA & Biological Data

The database is going to keep a record of all the DNA samples. This will include the results from the lab and the information that confirms what species it is. The DNA samples are connected to the animals profile. This is done using a computer program that matches the samples to the individual wildlife profile. The database will have all the information, about the DNA samples and the wildlife profiles. It will keep them linked together.

FR-04: Health & Veterinary Records

Users can keep track of their pets veterinary history. This includes: Vaccination records, Diagnosed conditions, Treatments given When keeping these records users must fill in: The date of the service, A brief summary of what happened. This helps users keep their pets medical history organized. They can see what vaccinations their pets have had what conditions they have been diagnosed with and what treatments they have received. The veterinary history is easy to understand.

FR-05: RESTful API Access

Users can keep track of their pets medical history. This includes: Vaccination records, Diagnosed conditions, Treatments given, When adding a record users must fill in: Date of service, A brief summary, The system has a way for other programs to access its data. This is done through: Endpoints for resources, like animal movements and species, Data is sent in a JSON format , A special token is required for every request to ensure authorized access. The token is used for authentication.

FR-06: User Management

Administrators can control user accounts. They can give users jobs and take away access if needed. Passwords are kept safe using bcrypt hashing. Each user has to verify their email address, which's unique, to them. The system is designed to keep everything.

FR-07: Search & Filter

The system has a good search feature that lets users find what they need by looking at the type of species, where something is located or when it happened. The system shows the search results one page at a time which makes it easier for users to look at the species information location details or timeframes they searched for.

FR-08: CSV Bulk Upload

Data Entry Operators can perform bulk movement data imports. The system validates every row in the file, committing valid data and generating a report for any skipped entries.

FR-09: Data Export

Researchers can get their search results in formats like JSON or CSV. They can do this in two ways. One way is through the website. The other way is through API endpoints. This helps them to easily use their search results. They can choose how they want to work with their results. The export option is available on the web and, through API. So researchers can export results in JSON or CSV. It is easy to use for them. They have choices to get results in formats.

2.5 Non-Functional Requirements

ID	Requirement	Description
NFR-01	Performance	API responses will be returned within 2 seconds for standard queries.
NFR-02	Scalability	The database shall handle up to 1 million animal records without performance loss.
NFR-03	Security	All API calls require JWT authentication. Passwords shall be hashed using bcrypt and also will use Salt method for better.
NFR-04	Availability	The system will be available 99% of the time after deployment.
NFR-05	Data Integrity	All records should have timestamps and audit logs showing who created or modified them.
NFR-06	Usability	There will be a simple web interface for non technical data entry operator to use without training.
NFR-07	Compatibility	The API will follow REST standards and work with any programming language or tool.

2.6 Use Cases

Use Case Diagram

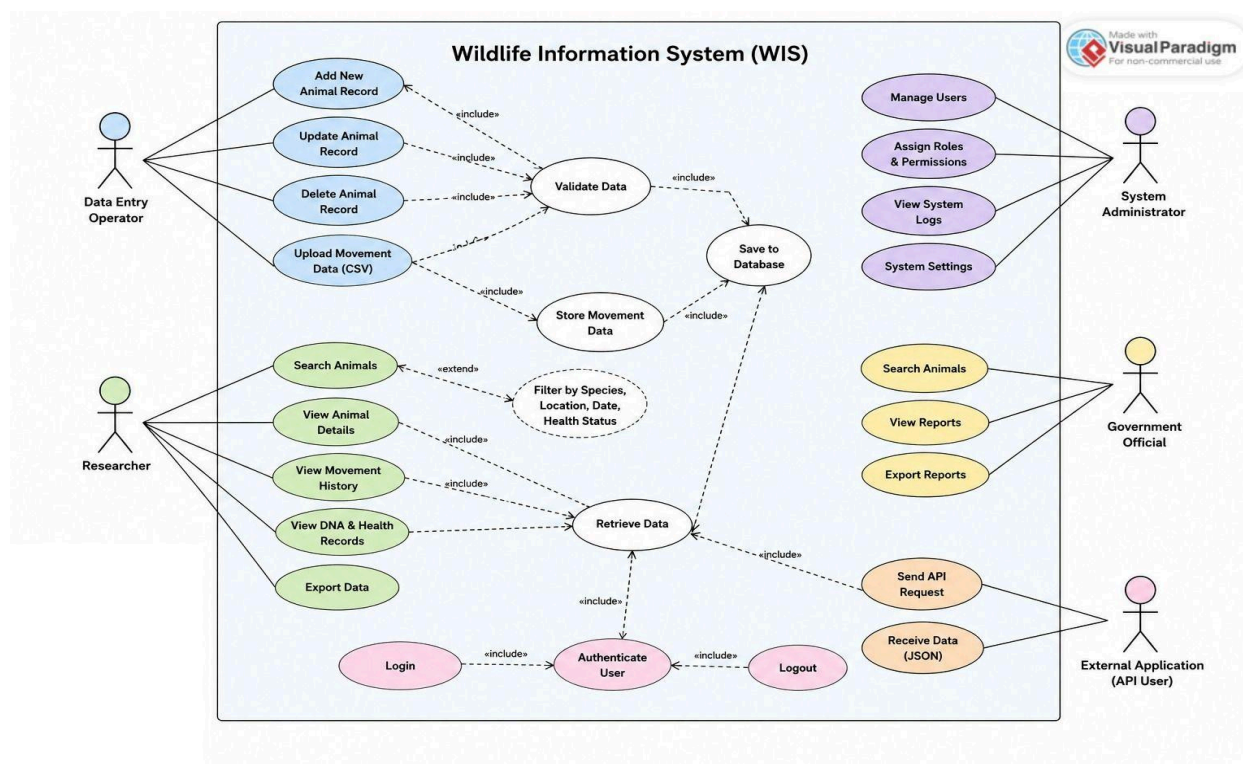


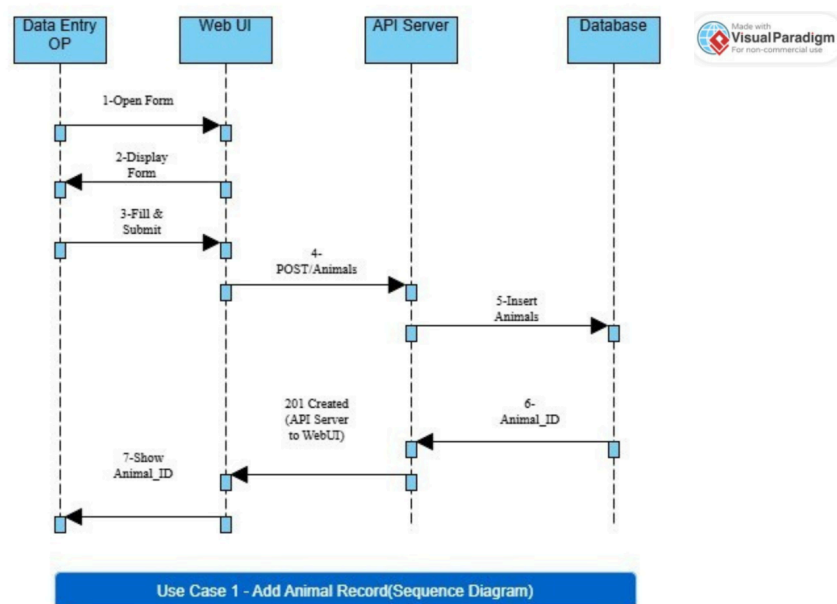
Figure 1: Use Case Diagram — Wildlife Information System

.1 Use Case Summary

UC ID	Name	Primary Actor	Category
UC-01	Add Animal Record	Data Entry Operator	Data Management
UC-02	Query Animal Data via API	Researcher / External App	API Access
UC-03	Upload CSV Movement Data	Data Entry Operator	Bulk Upload
UC-04	Search Animals	Researcher / Gov. Official	Search & Filter
UC-05	Manage Users	Admin	User Management
UC-06	View Health Records	Researcher / Vet	Data Access
UC-07	Export Data	Researcher	Data Export
UC-08	User Login / Authentication	All Users	Authentication
UC-09	Add DNA Sample Record	Data Entry Operator	Data Management

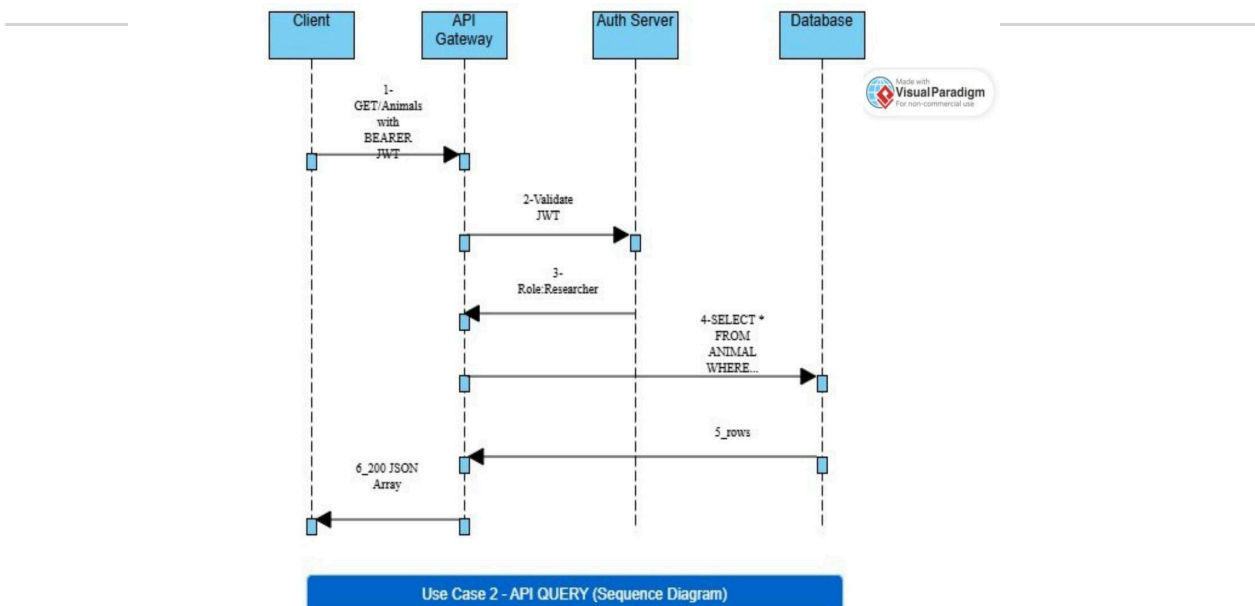
UC-01: Add Animal Record

Use Case ID	UC-01
Name	Add Animal Record
Actor	Data Entry Operator
Pre-condition	User is logged in with Data Entry or Admin role.
Post-condition	New animal record saved in the Animals table with a unique ID.
Main Flow	1. User opens Add Animal form. 2. System displays required fields. 3. User fills in species, name, gender, age, weight, location and submits. 4. System validates all fields. 5. System saves record. 6. System returns success message with new Animal ID.
Alternate Flow	If required fields are missing, system returns HTTP 400 with field level error messages. User corrects and resubmits.

Diagram: Add Animal Record => Sequence Diagram

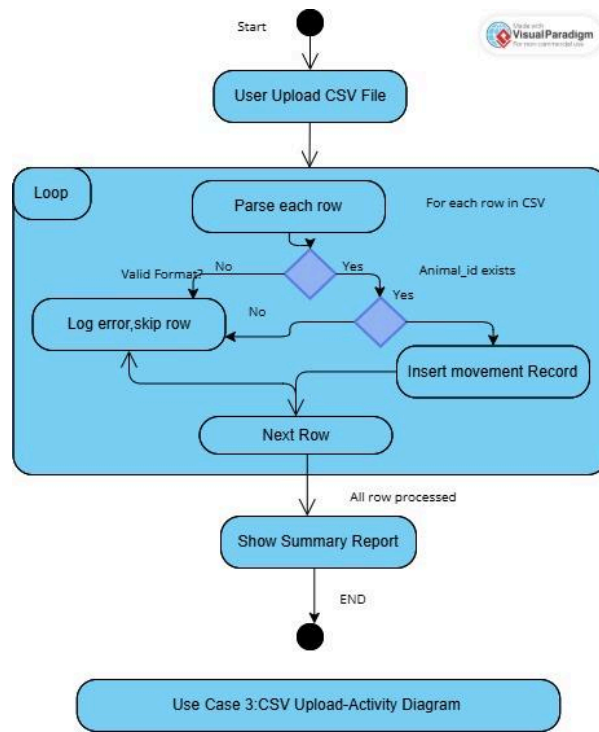
UC-02: Query Animal Data via API

Use Case ID	UC-02
Name	Query Animal Data via API
Actor	Researcher / External Application
Pre-condition	Client has a valid JWT token.
Post-condition	System returns JSON array of matching animal records with HTTP 200.
Main Flow	1. Client sends GET /api/animals?species=Snow+Leopard with Bearer token. 2. API Gateway validates JWT. 3. Auth Service confirms token and returns user role. 4. Query Controller passes parameters to Animal Service. 5. Animal Service executes SQL query. 6. Results formatted as JSON. 7. HTTP 200 returned with payload.
Alternate Flow	Invalid/expired token: HTTP 401 Unauthorized. Missing required query params: HTTP 400 Bad Request. No results found: HTTP 200 with empty array [].

Diagram: API Query => Sequence Diagram

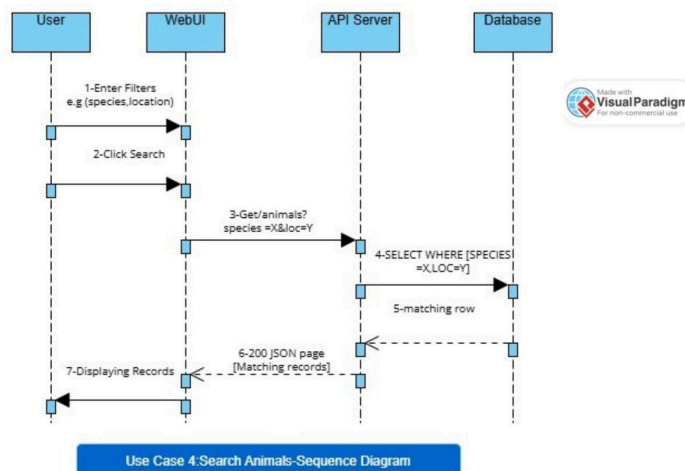
UC-03: Upload CSV Movement Data

Use Case ID	UC-03
Name	Upload CSV Movement Data
Actor	Data Entry Operator
Pre-condition	User logged in. CSV file correctly formatted (animal_id, lat, lng, timestamp).
Post-condition	Valid rows saved as Movement Records linked to animal IDs. Upload report shown.
Main Flow	1. User navigates to Upload Movement Data page. 2. User selects and uploads CSV file. 3. System reads file and validates each row. 4. Valid rows inserted into Movement_Records table. 5. Summary report displayed: total rows, saved count, skipped rows with reasons.
Alternate Flow	Rows with missing/invalid coordinates are skipped. Rows with non-existent animal IDs are flagged. User sees per-row error report.

Diagram: CSV Upload => Activity Diagram

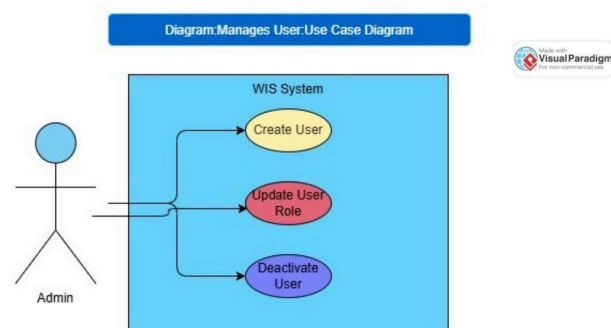
UC-04: Search Animals by Species or Location

Use Case ID	UC-04
Name	Search Animals by Species or Location
Actor	Researcher / Government Official
Pre-condition	User is logged in (any role).
Post-condition	Filtered, paginated list of matching animal records displayed.
Main Flow	1. User enters species name or location in search box. 2. User optionally sets date range filter. 3. User clicks Search. 4. System builds query with filters. 5. Database returns matching rows. 6. System displays paginated results (20 per page).
Alternate Flow	No records found: system displays 'No records found' message. If date range is invalid (end before start), system shows validation error.

Diagram: Search Animals => Sequence Diagram

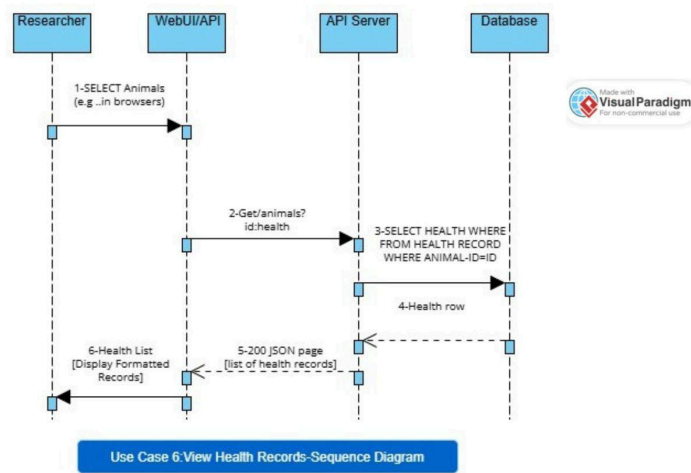
UC-05: Manage Users (Admin)

Use Case ID	UC-05
Name	Manage Users (Admin)
Actor	System Administrator
Pre-condition	User is logged in as Admin.
Post-condition	New user created or existing user role/status updated in Users table.
Main Flow	1. Admin opens User Management panel. 2. Admin clicks Create User or selects existing user. 3. Admin fills in name, email, role. 4. System validates email uniqueness. 5. System saves user; password hash generated. 6. Confirmation message shown.
Alternate Flow	Duplicate email: system returns error 'Email already exists'. Admin corrects and resubmits.

**Diagram: Manage Users (Admin) => Use Case Diagram**

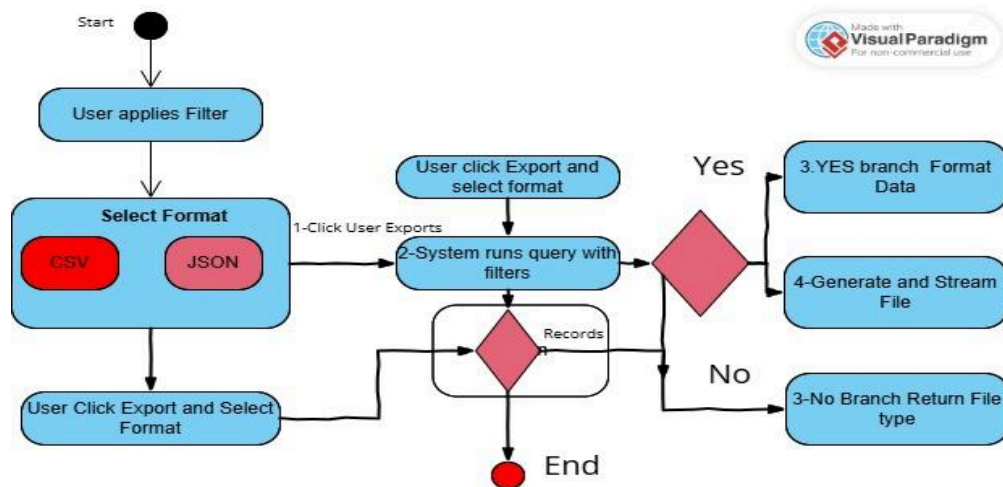
UC-06: View Health Records

Use Case ID	UC-06
Name	View Health Records
Actor	Researcher / Veterinarian
Pre-condition	User logged in with Researcher or Admin role. Animal ID known.
Post-condition	Health record list for the selected animal returned.
Main Flow	1. User opens Animal Detail page or sends GET /api/animals/:id/health. 2. System retrieves all health records for that animal ID. 3. Records sorted by date descending. 4. System returns list with condition, treatment, vet name, vaccination status.
Alternate Flow	Animal ID not found: HTTP 404. No health records exist: returns empty array with HTTP 200.

Diagram: View Health Records => Sequence Diagram

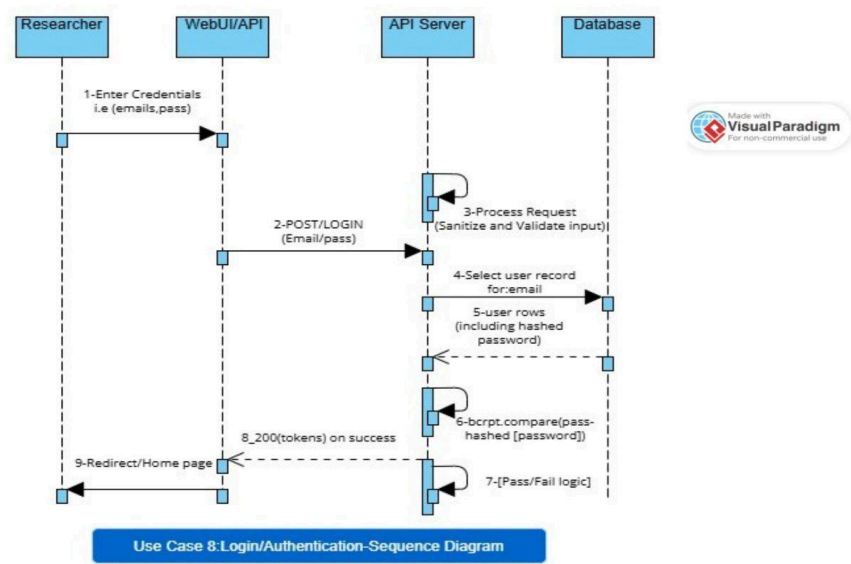
UC-07: Export Data

Use Case ID	UC-07
Name	Export Data
Actor	Researcher
Pre-condition	User logged in. Search/filter applied (optional).
Post-condition	CSV or JSON file downloaded by the user.
Main Flow	1. User performs search (optional). 2. User clicks Export and selects format (CSV or JSON). 3. System executes the query with current filters. 4. System formats data and generates file. 5. File downloaded to user's device.
Alternate Flow	No records match filters: system exports empty file with headers only and notifies user.

Diagram: Export Data => Activity Diagram**Diagram: Export Data:Activity Diagram**

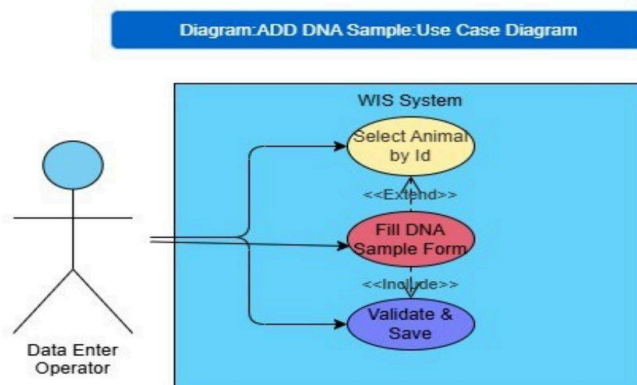
UC-08: User Login / Authentication

Use Case ID	UC-08
Name	User Login / Authentication
Actor	All Users
Pre-condition	User has a registered account in the system.
Post-condition	User receives a valid JWT token used for subsequent API calls.
Main Flow	1. User opens login page or sends POST /api/auth/login with email and password. 2. System looks up user by email. 3. System compares password with bcrypt hash. 4. If match, system generates JWT with user role embedded. 5. Token returned with expiry time.
Alternate Flow	Wrong password: HTTP 401 Unauthorized. Non-existent email: HTTP 401 (generic, no user enumeration). Account deactivated: HTTP 403 Forbidden.

Diagram: Login / Authentication => Sequence Diagram

UC-09: Add DNA Sample Record

Use Case ID	UC-09
Name	Add DNA Sample Record
Actor	Data Entry Operator
Pre-condition	User logged in with Data Entry or Admin role. Animal ID exists in the system.
Post-condition	DNA sample record saved and linked to the specified animal.
Main Flow	1. User opens DNA Records section and selects an animal. 2. User fills in: sample ID, collection date, lab name, result, species confirmed flag. 3. User submits form. 4. System validates required fields and verifies animal ID. 5. System saves DNA record. 6. Confirmation with sample ID shown.
Alternate Flow	Animal ID not found: HTTP 404. Duplicate sample ID: HTTP 409 Conflict. Missing required fields: HTTP 400 with field errors.

Diagram: Add DNA Sample => Use Case Diagram

3. ANALYSIS

3.1 Domain Model

The domain model identifies the main real world concepts or entities in the Wildlife Information System and how they relate to each other.

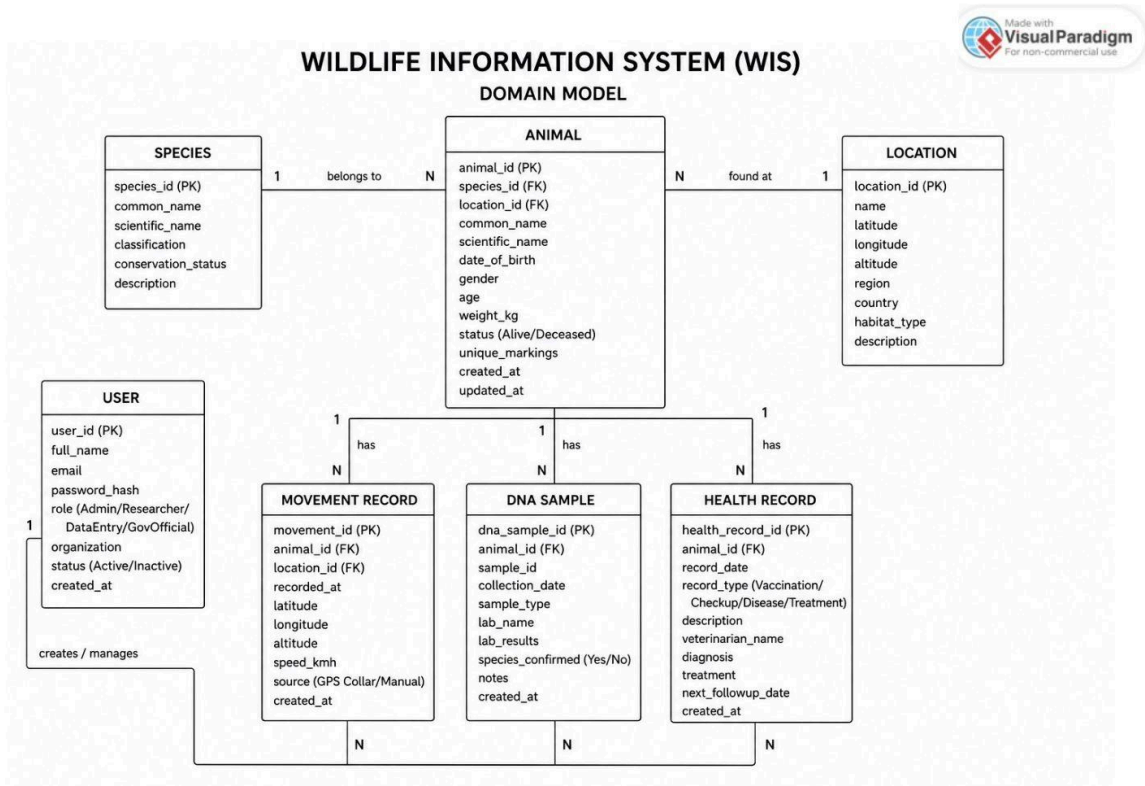


Figure 2: Domain Model — Key Entities and Relationships

Key entities: Animal (central), Species, Movement Record, DNA Sample, Health Record, User, and Location. One Animal has many Movement Records, Health Records, and DNA Samples. Each Animal belongs to one Species.

3.2 Data Flow Diagram (DFD)

Level 0 DFD (Context Diagram)

The Context Diagram shows the system as a single process and identifies all external entities that interact with it.

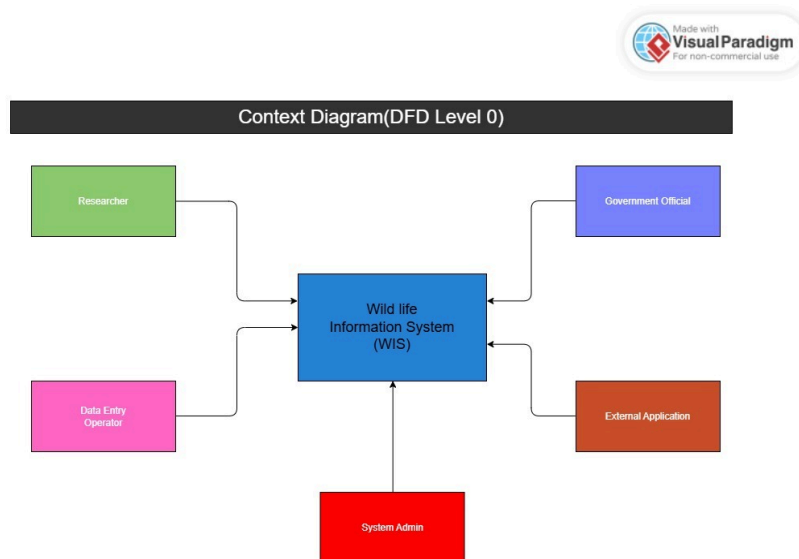


Figure 3: Context Diagram (DFD Level 0)

External Entities: Researcher, Data Entry Operator, System Administrator, Government Official, External Application (through API). The system obtains data about animals, movement details, DNA samples, and animal health information from the Data Entry Operator. Information is requested by the Researcher and Government Officials.

Level 1 DFD

The Level 1 DFD breaks the system into main processes:

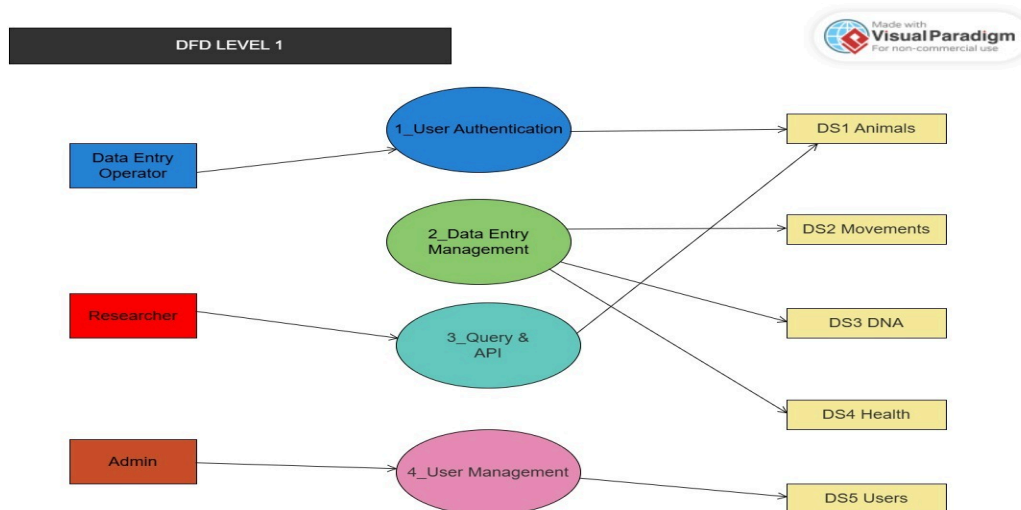


Figure 4: DFD Level 1 — System Processes

Process 1 - User Authentication: Responsible for log-in, token creation, and user-role authorization.

Process 2 - Data Entry Processing: Accepts and processes inputted data on wildlife, including animal details, movement, DNA, and health data.

Process 3 - Data Search & API Request Processing: Accepts requests from researchers or other applications and delivers requested data from the database.

Process 4 - User Processing: An admin user can create, update, or delete user account information.

Data Storage: DS1 - Animals' table, DS2 - Movement's table, DS3 - DNA's table, DS4 - Health's table, DS5 - Users' table.

3.3 System Sequence Diagrams

The following describe the interaction between actors and the system for key use cases:

Sequence 1: Add Animal Record

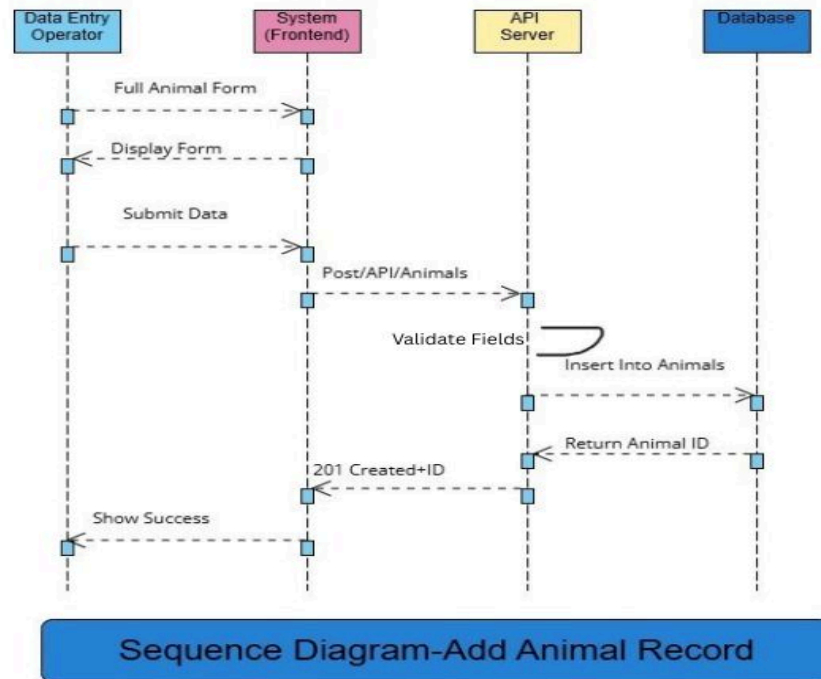


Figure 5: Sequence Diagram — Add Animal Record

- Add Animal Form is launched by Data Entry Operator.
- System displays the form with mandatory fields.
- Details are entered by Operator and Submit button is pressed.
- System validates all mandatory fields.
- System saves the record in Animals database table.
- System responds with success message containing the unique animal ID.

Sequence 2: API Query by Researcher

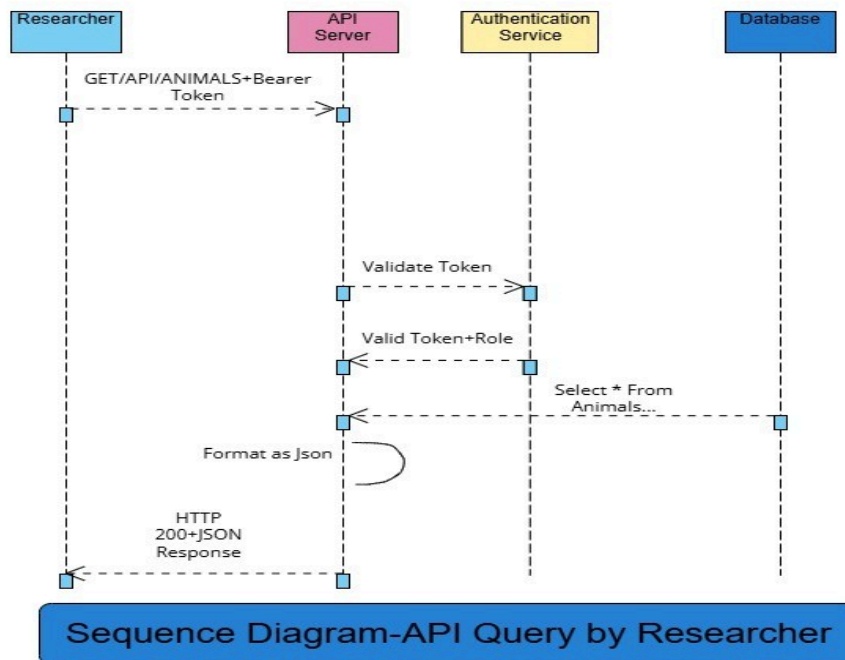


Figure 6: Sequence Diagram — API Query by Researcher

HTTP GET request containing query parameters along with the Bearer Token is sent by the researcher.

Auth Service authenticates the token.

Query is processed by the system on the basis of database.

Response from the system is in the form of JSON data and HTTP status code 200.

3.4 Activity Diagram

The following activity describes the flow when a researcher queries data through the API:

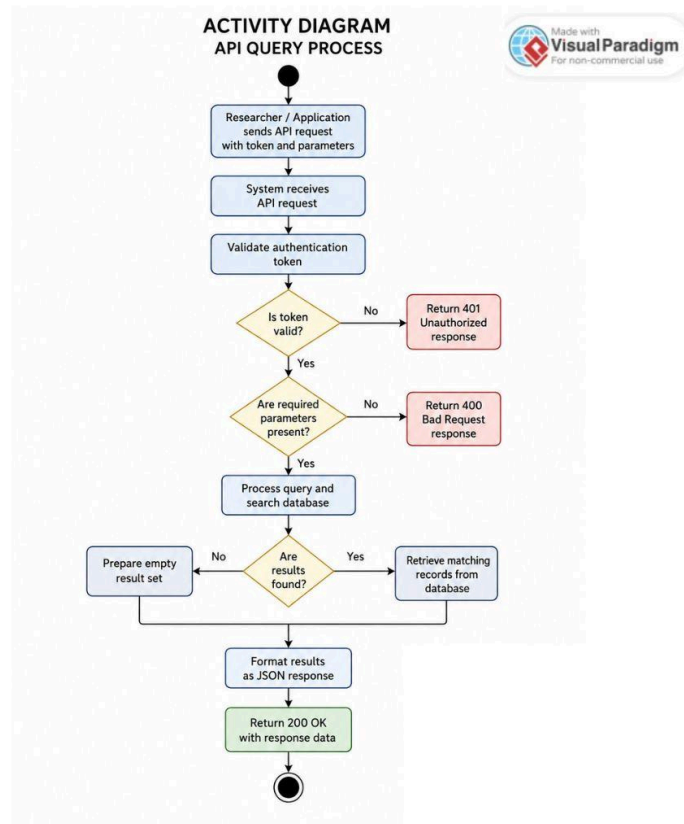


Figure 7: Activity Diagram — API Query Flow

Begin: Researcher sends API request along with token.

Decision: Is the token valid? If No → Send 401 response. If Yes → Proceed further.

Decision: Are all necessary parameters included in the query? If No → Send 400 response. If Yes → Continue.

The system searches the database for the data. If not found → Return an empty array. If found → Format the response in JSON.

System sends 200 response. End.

3.5 Database ER Diagram

The Entity-Relationship diagram shows the complete database schema for the Wildlife Information System, including all tables, primary keys (PK), foreign keys (FK), and their relationships.

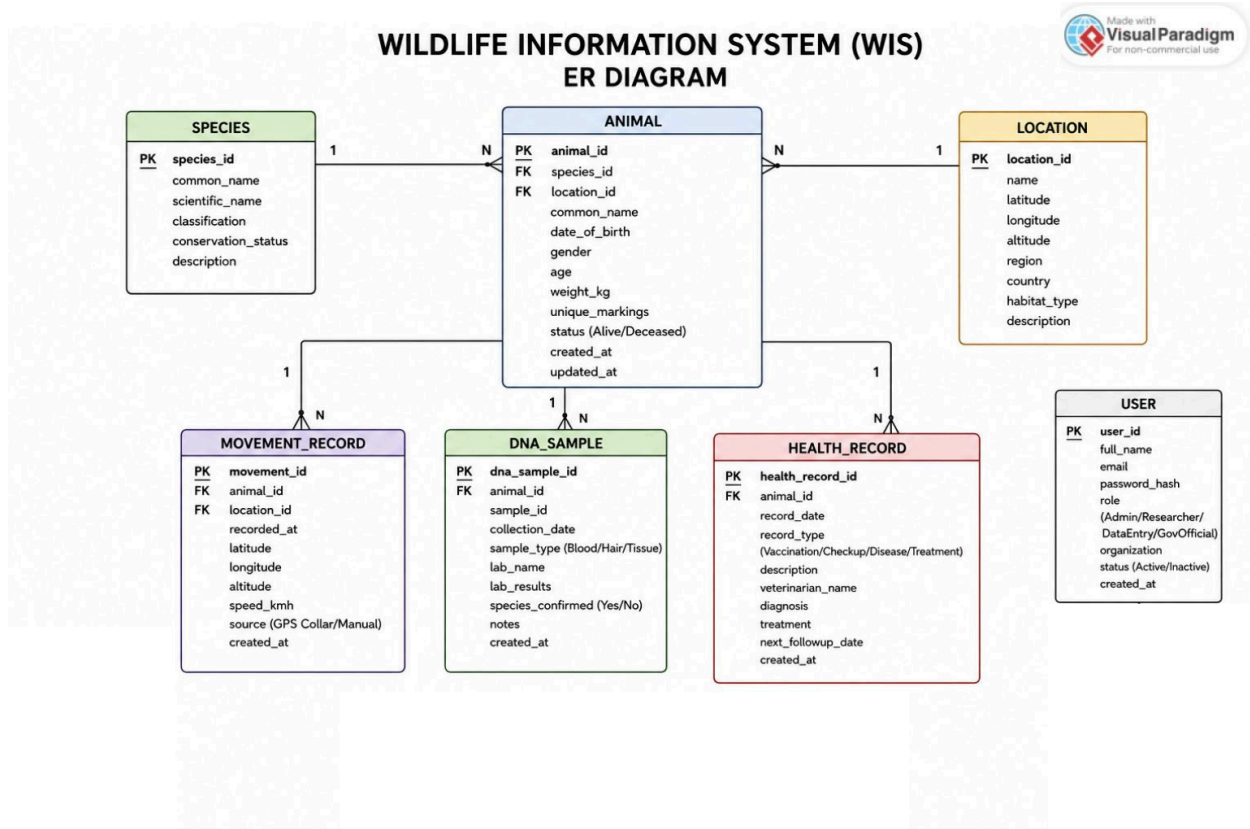


Figure 8: ER Diagram — Database Schema

Key relationships: Animal references Species (FK). Movement_Record, DNA_Sample, and Health_Record all reference Animal (FK). Each User manages multiple operations tracked in audit logs.

4. DESIGN

4.1 Class Diagram

The class diagram represents the main classes in the system with their attributes and methods:

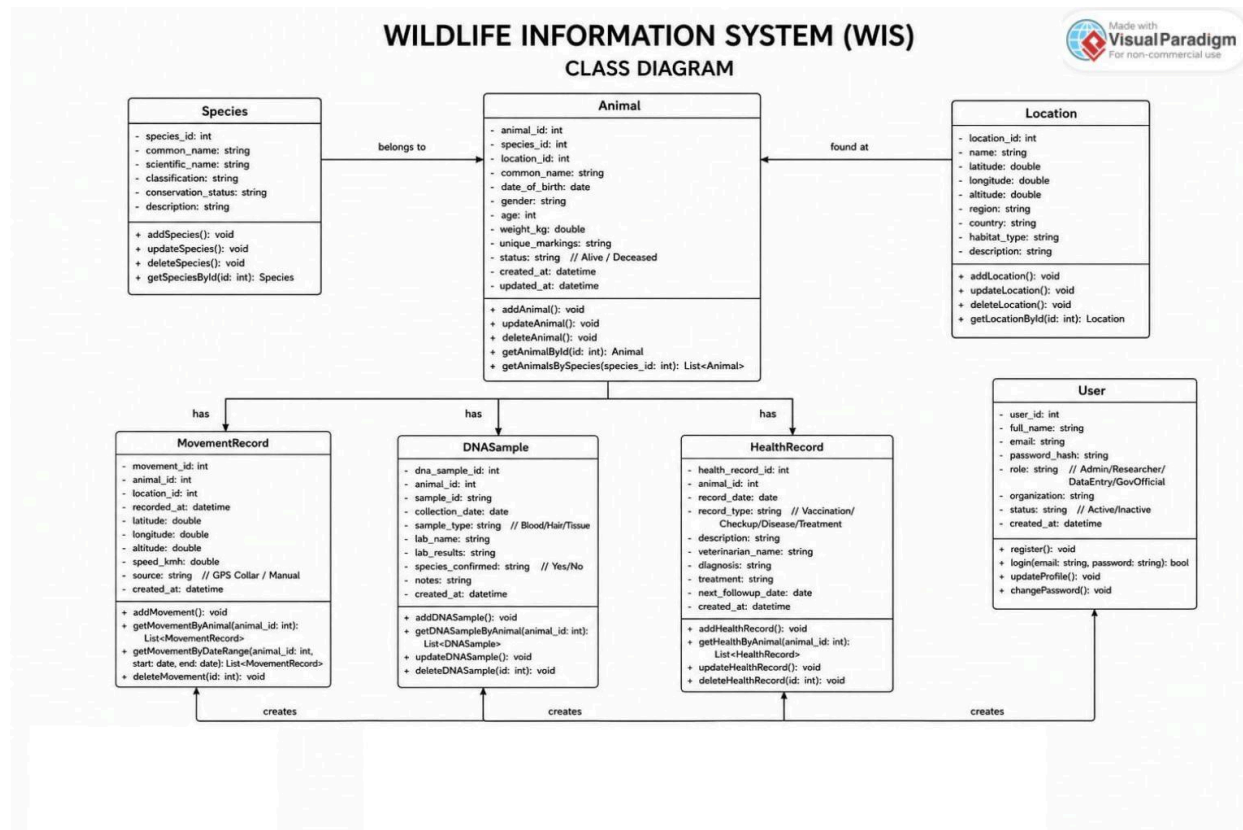


Figure 9: Class Diagram

Class: Animal

Attributes: animalId, speciesId, name, gender, ageYears, weightKg, locationId, addedBy, addedAt

Methods: save(), update(), delete(), getMovementHistory(), getHealthRecords(), getDNASamples()

Class: MovementRecord

Attributes: recordId, animalId, latitude, longitude, recordedAt, source

Methods: save(), getByAnimal(), bulkInsert()

Class: DNASample

Attributes: sampleId, animalId, collectedAt, labName, result, speciesConfirmed

Methods: save(), getByAnimal()

Class: HealthRecord

Attributes: healthId, animalId, date, condition, treatment, vetName, vaccineGiven

Methods: save(), update(), getByAnimal()

Class: User

Attributes: userId, name, email, passwordHash, role, createdAt

Methods: login(), logout(), updateProfile(), changePassword()

Class: Species

Attributes: speciesId, commonName, scientificName, conservationStatus, habitat

Methods: save(), getAll(), getName()

4.2 Sequence Diagram

The detailed sequence for "Query Animal Data via API" is described below:

1. External App sends: GET /api/animals?species=Snow+Leopard with Authorization: Bearer <token>
2. The API Gateway validates the JWT token.
3. The Auth Service validates the token and provides the user role.
4. The Query Controller handles the incoming request and forwards the parameters to the Animal Service.
5. The Animal Service constructs an SQL query that queries both the animals table and the species table.
6. The database queries the relevant rows.
7. The Animal Service returns the result as JSON.
8. The API Gateway responds with an HTTP 200 and the JSON payload.

4.3 Architecture

The Wildlife Information System follows a 3-Tier Architecture:

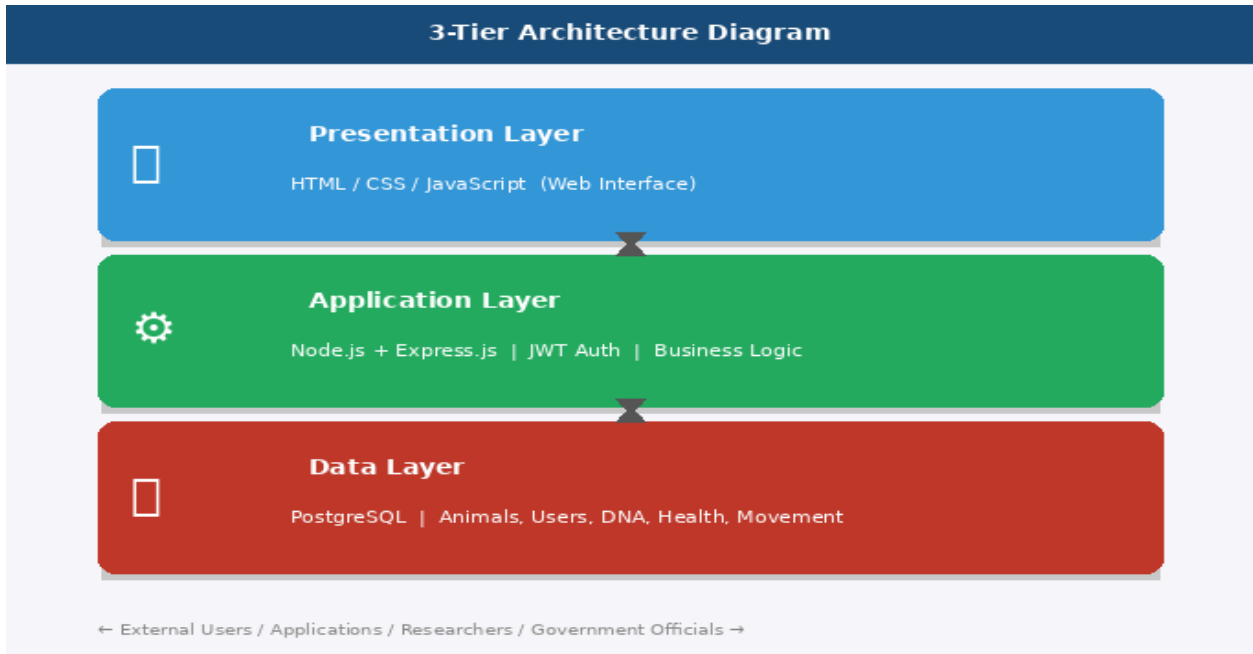


Figure 10: 3-Tier Architecture Diagram

Layer	Component	Responsibility
Presentation Layer	HTML/CSS/JS Web Interface	Basic UI for data entry, search, and export
Application Layer	Node.js + Express.js API Server	Business logic, authentication, query processing
Data Layer	PostgreSQL Database	Stores all wildlife records, users, and audit logs

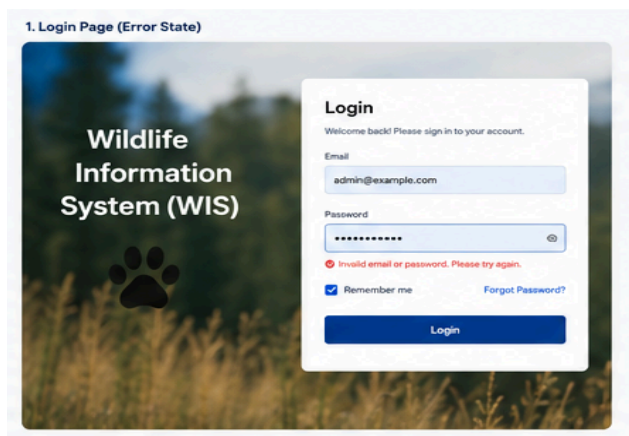
External users and applications communicate with the Application Layer through the RESTful API. The Application Layer handles authentication, business logic, and data formatting. The Data Layer stores everything persistently.

Key API Endpoints

Method	Endpoint	Description
GET	/api/animals	Get list of all animals (with filters)
GET	/api/animals/:id	Get a single animal by ID
POST	/api/animals	Add a new animal record
PUT	/api/animals/:id	Update animal record
DELETE	/api/animals/:id	Delete animal record
GET	/api/animals/:id/movements	Get movement history for an animal
POST	/api/movements/upload	Bulk upload CSV movement data
GET	/api/animals/:id/health	Get health records for an animal
GET	/api/animals/:id/dna	Get DNA records for an animal
POST	/api/auth/login	User login and get JWT token
GET	/api/species	Get list of all species

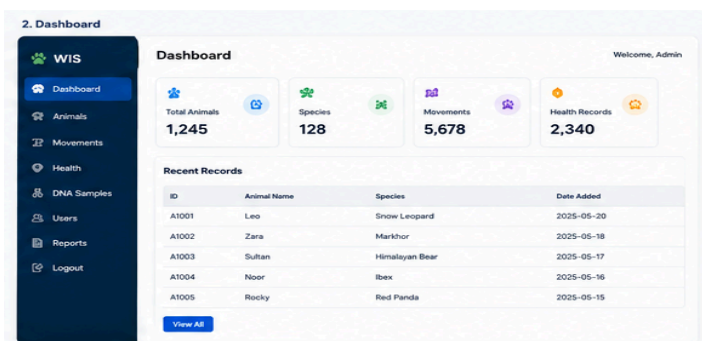
4.4 Interface Design

This application would have an easy-to-use web-based interface. The key pages include:

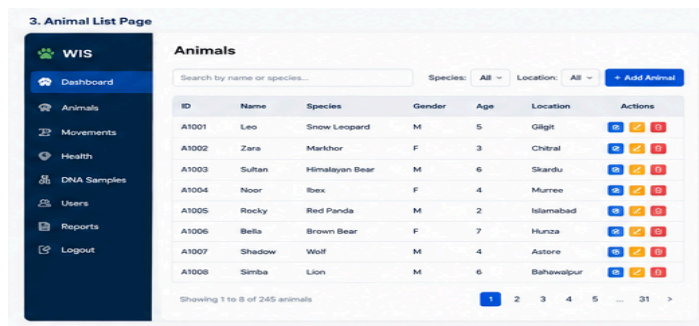


Login Page : Contains the email address field, password field, and login button. Displays an error message if incorrect.

Dashboard : Has a welcome message, statistics about animals (including total number, number of species, and some recent records), and navigation menu.



Animal List Page : Displays a sortable table containing all animals with View, Edit, and Delete buttons.



Add Animal Form : Contains fields for entering information on the species, name, gender, age, weight, and location of an animal. Includes a submission button for recording the data.

Movement Records Page: Displays a sortable table containing GPS coordinates and the time stamps for a particular animal with an option to add a CSV file.

Record ID	Latitude	Longitude	Recorded At
MS001	35.1234	74.5678	2025-05-20 08:10:30
MS002	35.2234	74.6678	2025-05-20 12:45:20
MS003	35.3234	74.7678	2025-05-20 16:10:50
MS004	35.4234	74.8678	2025-05-20 20:30:15
MS005	35.5234	74.9678	2025-05-21 00:50:05

API Documentation Page: A straightforward page listing APIs, required parameters, and example responses.

User Management (Admin Only): Displays a sortable table containing information on all the users registered with options to create or remove an account.

Design principles: Minimal use of color; the only colors being used are blue and white. All buttons have descriptive names. Errors are displayed in red. Success messages are displayed in green.

6. API Documentation Page

API Documentation

Base URL
https://api.wis.com/v1

Endpoints

Method	Endpoint	Description
GET	/animals	Get list of all animals
GET	/animals/{id}	Get details of a specific animal
POST	/animals	Create a new animal
PUT	/animals/{id}	Update an existing animal
DELETE	/animals/{id}	Delete an animal

Example: Get Animal Details

GET /animals/{id}

Parameters

Name	Type	Description
id	integer	Animal ID

Response (200 OK)

```
{  "id": "A1001",  "name": "Leo",  "species": "Snow Leopard",  "gender": "m",  "age": 5,  "weight": 45,  "location": "Hillgit"}
```

7. User Management (Admin Only)

WIS

Dashboard

Animals

Movements

Health

DNA Samples

Users

Reports

Logout

Users

+ Create User

ID	Full Name	Email	Role	Status	Created At	Actions
U001	Admin User	admin@example.com	Administrator	Active	2025-01-10 09:30:00	
U002	Ranger One	ranger1@example.com	Ranger	Active	2025-01-12 10:15:20	
U003	Vet Doctor	vet@example.com	Veterinarian	Active	2025-01-14 11:45:10	
U004	Researcher	research@example.com	Researcher	Inactive	2025-01-18 14:20:30	
U005	Viewer	viewer@example.com	Viewer	Active	2025-01-20 16:05:40	

Showing 1 to 5 of 25 users

1

2

3

4

5

..

Create User

Full Name

Enter full name

Email

Enter email

Role

Select Role

Password

Enter password

Confirm Password

Confirm password

Status

Active

Create User

Cancel

Edit User (U002)

Full Name

Ranger One

Email

ranger1@example.com

Role

Ranger

Status

Active

Update User

Cancel

5. IMPLEMENTATION

Remark: It should be noted that the implementation stage has not yet been undertaken in the context of this assignment. System design, database model, and the API structure have all been completely designed. In future work, coding will be done using the designs mentioned in Section 4.

6. TESTING

6.1 Acceptance Testing

Acceptance tests ensure that the system satisfies the requirements set at the beginning of the project. The objective is to verify that the system delivers what the stakeholders desire before its delivery or usage.

The acceptance criteria are as follows:

- The users should be able to log into the system and get an access token.
- The authorized user should add a new animal and retrieve it using the API.
- The CSV file containing movement information should be able to upload and saved.
- The search filters should return the right animals according to species or geographical position.
- API should return proper HTTP error messages (401, 400, 404).
- The admin should be able to create a new user with the role.

6.2 Test Cases

TC ID	Feature	Input	Expected Output
TC-01	User Login	Valid email and password	JWT token returned, status 200
TC-02	User Login (wrong)	Valid email, wrong password	Status 401 Unauthorized
TC-03	Add Animal	Valid animal data fields	Record saved, new ID returned
TC-04	Add Animal (missing)	Species name missing	Status 400, field error message
TC-05	Get Animal by ID	Valid animal ID in URL	Full animal record in JSON
TC-06	Get Animal (wrong ID)	ID that does not exist	Status 404 Not Found
TC-07	Search by Species	species=Snow Leopard query	List of matching animals
TC-08	CSV Upload (valid)	Correctly formatted CSV file	All rows saved successfully
TC-09	CSV Upload (bad)	CSV with missing coordinates	Rows skipped, error report shown
TC-10	API without a token	GET request without Auth header	Status 401 Unauthorized
TC-11	Export Data	Request to export search results	JSON or CSV file downloaded
TC-12	Admin creates user	New user details with role	User account created successfully

6.3 Test Plan

Testing Approach: Black Box Testing

The black box testing approach will be used as the main testing technique by the development team.

Tests are created solely based on input values and expected output results.

Black box testing is appropriate for this case due to the following reasons:

- There are specific inputs and expected outputs of API endpoints.
- Functional requirements will be validated without having to read the code.
- This reflects the way actual users and other applications will work with the product.

Test Levels

Unit Testing: Unit tests will be conducted for each API endpoint with the help of Postman. In each unit test, only one API endpoint is tested.

Integration Testing: Integration testing will be carried out on the completion of development of all the API endpoints, and they will be tested together.

System Testing: System testing will involve conducting tests for the entire application system from logging in to exporting the data via APIs.

Acceptance Testing: The final stage of testing that involves testing against the requirements specification.

Test Tools

Postman: For manual API testing.

Browser DevTools: For testing the web interface.

Test Schedule

Phase	Test Type	When
After each endpoint	Unit Test (Postman)	During development
After full API build	Integration Test	After Increment 2
Before submission	System & Acceptance Test	Final week

Definitions & Acronyms

Term / Acronym	Definition
WIS	Wildlife Information System
API	Application Programmable Interface
REST	Representational State Transfer
JWT	JSON Web Token
FR	Functional Requirement
NFR	Non-Functional Requirement
UC	Use Case
CRUD	Create, Read, Update, Delete
CSV	Comma-Separated Values
JSON	JavaScript Object Notation
DFD	Data Flow Diagram
ER	Entity-Relationship